



Arduino: Sensors, Neopixels & Music

by Henry Li, based on EE1000 Lab Sessions by Prof. David Howard



Introduction

In this workshop session we will see how to use an Arduino to sense light levels using a Light Dependent Resistor (LDR), to vary the colours and brightness on Neopixels, to measure distance to an object using an Ultrasonic Range Sensor, and to play simple music with a buzzer.

Component & Equipment List

Component	Quantity
Light Dependent Resistor (LDR)	1
100Ω Resistor	1
10kΩ Resistor	1
WS2812 Neopixels Module	1
HC-SR04 Ultrasonic Range Sensor	1
2V Piezoelectric Buzzer	1

Equipments	Quantity
Arduino Uno	1
USB-A to USB-B Cable	1
Wires	15

Software	
Arduino IDE	-

Complete Circuit

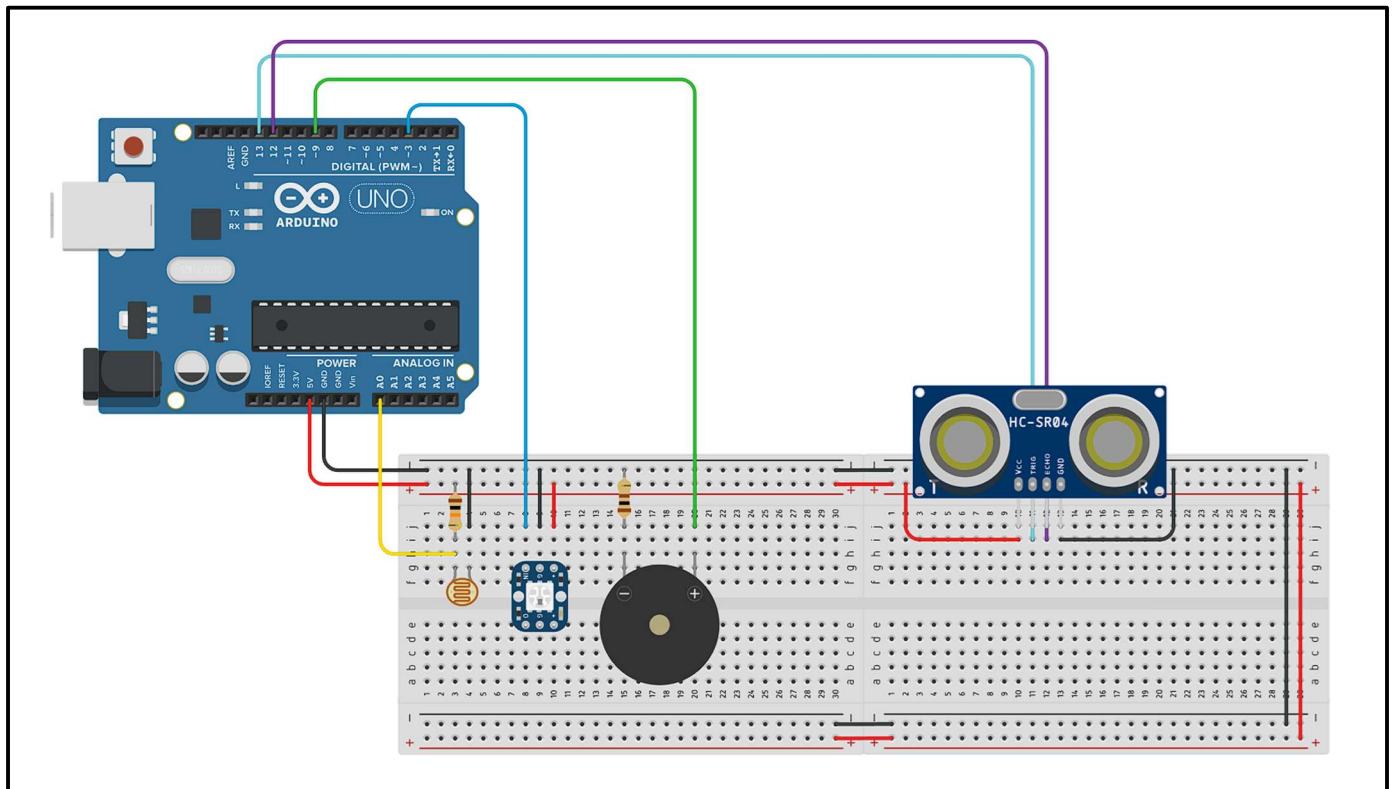


Fig.1: Circuit Design, designed with Autodesk Tinkercad.



Component Pinouts & Schematics

Component	Pin
LDR	A0 / GND
10k Ω Resistor	A0 / +5V
WS2812 Neopixels	
Power (+V)	+5V
Signal (D+ / IN / SIG)	D3
Ground (GND)	GND
HC-SR04 Ultrasonic Sensor	
Power (+V)	+5V
Trigger (TRIG)	D13
Echo	D12
Ground (GND)	GND
Piezo Buzzer & 100Ω Resistor	
D9 / GND	

WS2812 Neopixels

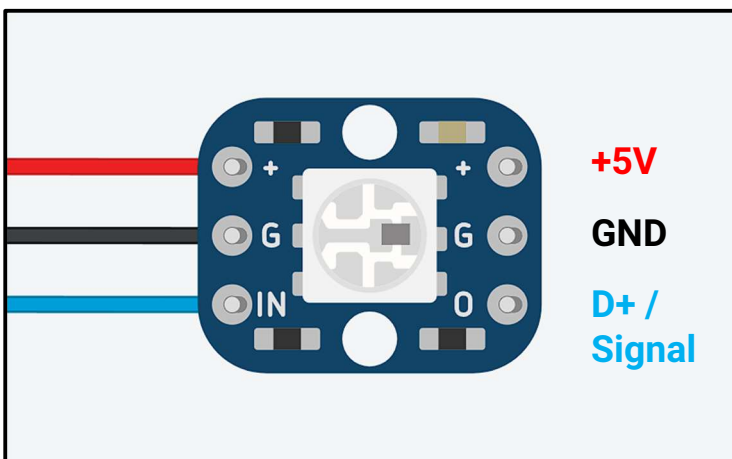


Fig.2: Pinout of WS2812 Single Unit Neopixels, created with Autodesk TinkerCAD.



1. Light Sensing & Neopixels

In this section, we will be using the LDR and the Neopixels module.

A Light Dependent Resistor, LDR, is basically a variable resistor with the resistance varying based on the brightness level of the environment. Therefore it is sometimes called a photoresistor.

A Neopixel™ is a specialised miniature LED unit that contains three (sometimes four) individual LEDs (Red, Green, Blue and sometimes White). It is also programmable to control each LED element separately for colour and brightness.

The Arduino IDE has a number of pre-coded libraries and examples for various peripherals and tasks and has some examples specifically for getting to grips with various common functions.

Neopixels are easy to program but they require a specific library which you may need to install. Go to

Sketch >> Include Library >> Manage Libraries, then search for "NeoPixel"

Select the latest version of "Adafruit NeoPixel by Adafruit" by clicking on the drop-down menu next to the Install button.

a. Light Sensing with LDR & Brightness Variation with Neopixels

The voltage produced by the LDR is being input A0, so we can read this value and use it to control the brightness of an LED, or even Neopixels.

Firstly, write these setup codes:

```
#include <Adafruit_NeoPixel.h>
#define PIN 9 // Neopixels pin
#define NUMPIXELS 1 // Number of Neopixels

int brightness = 0; // Brightness of Neopixels
int ldr = 0; // Voltage from LDR

Adafruit_NeoPixel pixels = Adafruit_NeoPixel(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);

void setup() {
  // Initialise NeoPixel library
  pixels.begin();

  // Sets pixel default as white
  pixels.setPixelColor(0, pixels.Color(255, 255, 255));
}
```

These codes will set up the Neopixel library and all of the variables in this section. Feel free to change the default RGB colour code within the "pixels.Color" function to whatever colour you prefer.

Then we will use the "analogRead" command, which returns integer values from 0 to 1023. We can then use the "map" function to adjust the range in the variable 'brightness' accordingly, since a PWM output (the pin of the Neopixel) can only handle values from 0 to 255.



Write these codes into the “loop” function:

```
void loop() {
  // set the brightness of pin 9
  ldr = analogRead(A0); // Read voltage from LDR

  // to map range of 'brightness' as 0 to 255 from 'ldr' of range 0 to 1023
  brightness = map(ldr, 0, 1023, 0, 255);

  for (brightness = 0; brightness <= 255; brightness++) {
    pixels.setBrightness(brightness); // Sets brightness
    Serial.print(ldr);
    Serial.println('\n');

    pixels.show(); // Output data to pixel
    delay(50); // Delay in ms
  }
}
```

b. Light Sensing with LDR & Colours Variation with Neopixels

Now we will move on to varying the colours on the Neopixel. You will have to create a new sketch for each section.

The setup codes are basically the same as above for setting up the Neopixel library, the only differences are the variables being declared for this section. And again, please feel free to change the default RGB colour code within the “pixels.Color” function to whatever colour you prefer, these preset colour codes are just some common used colours.

Write these setup codes:

```
#include <Adafruit_NeoPixel.h>
#define PIN 9 // Pin connected to the Neopixels
#define NUMPIXELS 1 // Number of Neopixels attached to the Arduino

Adafruit_NeoPixel pixels = Adafruit_NeoPixel(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);

int r = 0; // Red level
int g = 0; // Green level
int b = 0; // Blue level
int ldr = 0; // Analogue input
int colour = 0; // Colour choices

void setup() {
  pixels.begin(); // Initialise NeoPixel library
}
```

The code within the “loop” function is just based on a very simple logic compared to the previous section.

The code will first assigns the reading from the LDR to the variable “ldr”, then by using the “map” function we can adjust the 2 sets of data to match up. Once the map is stored to the variable “colour”, the program follows a simple ‘if... else if’ statement for different colours.

You are able to freely vary the maximum number of colours available by changing the last integer in the “map” function. You then just need to simply copy any of the “else if” statement, change the index of the colour and RGB colour code.



Finally, write these codes into the "loop" function:

```
void loop() {
  ldr = analogRead(A0); // Analogue Voltage from A0
  colour = map(ldr, 0, 1023, 1, 7); // Map voltage to colour

  if (colour == 1) { // Red
    r = 100; g = 0; b = 0;
  }
  else if (colour == 2) { // Green
    r = 0; g = 100; b = 0;
  }
  else if (colour == 3) { // Blue
    r = 0; g = 0; b = 100;
  }
  else if (colour == 4) { // Cyan
    r = 0; g = 100; b = 100;
  }
  else if (colour == 5) { // Purple
    r = 100; g = 0; b = 100;
  }
  else if (colour == 6) { // Yellow
    r = 100; g = 100; b = 0;
  }
  else if (colour == 7) { // White
    r = 100; g = 100; b = 100;
  }

  // Process & output RGB values from 0 to 255
  pixels.setPixelColor(0, pixels.Color(r, g, b)); // Set pixel colour
  pixels.show(); // This sends the updated pixel colour to the hardware
}
```

Challenge!

Try and swipe the **10kΩ resistor** to the **GROUND RAIL** and the **ground wire** for the LDR to the **POWER RAIL** and rerun the code to see what's gonna happen.

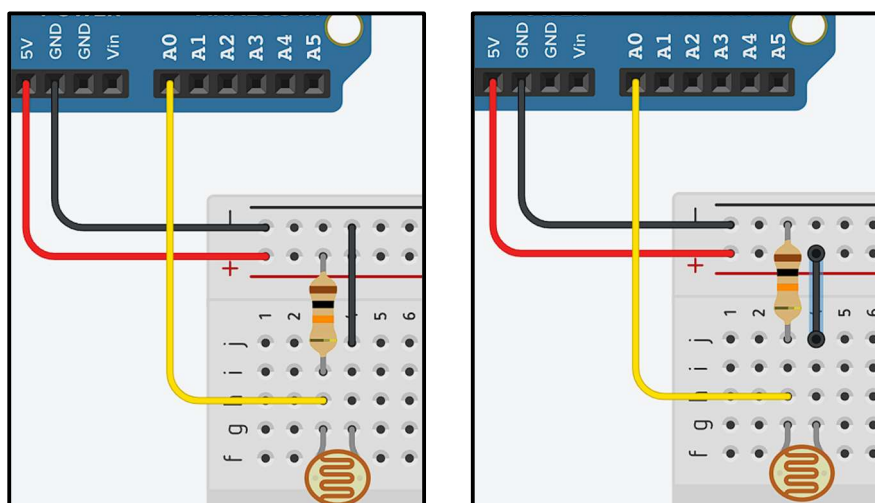


Fig.3 & Fig.4: LDR / 10kΩ Resistor Potential Divider – Before & After.

Now move onto the 'Ultrasonic Distance Sensing & Music' section if time allows.



2. Ultrasonic Distance Sensing & Basic Tone Music

In this section we will be measuring distance using an HC-SR04 distance sensor.

Figure 2 shows the pinout of the sensor: 5V, GND, Trigger and Echo. The sensor operates by emitting a 40kHz ultrasound pulses via 'Trigger' and 'listens' for a reflected pulse via 'Echo'. Its operating range is quoted as being 2cm to 4m.

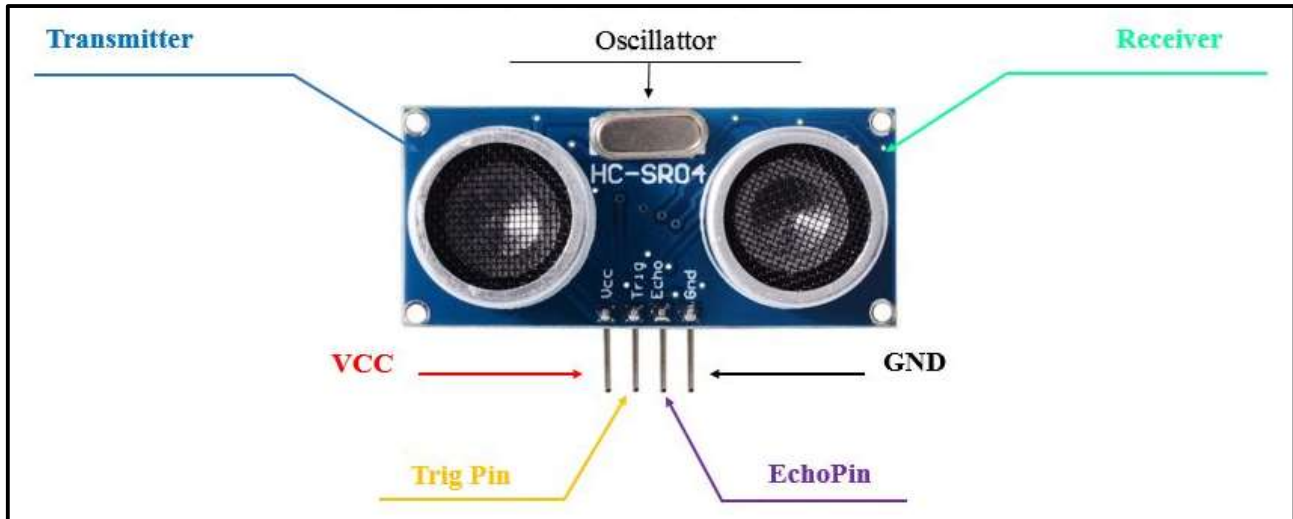


Fig.5: Pinout of HC-SR04 Ultrasonic Range Sensor. ^[1]

Firstly write these setup codes to setup the pins for the ultrasonic sensor:

```
#define trigPin 13
#define echoPin 12

void setup() {
  Serial.begin(9600);    // Setup Monitor
  pinMode(trigPin, OUTPUT); // Set Trigger as output
  pinMode(echoPin, INPUT);  // Set echo as input
}
```

a. Pulse Setup for Ultrasonic Range Sensor

The 'Trigger' is set HIGH for 10 μ s, which causes the ultrasonic burst of pulses to be sent out at the speed of sound (340 m/s).

The 'Echo' then emits a high pulse by using the "pulseIn" function whose length indicates the time of travel of the ultrasound pulse.

We can then program the Arduino to convert the returned 'echo' value into centimetres and display it on the serial monitor, or use the data for activating other outputs.

The next step is to convert the duration into distance (in metres), and then divide by 10000 to get it to centimetres with this line of code:

$$\text{distance} = (340.0 * (\text{duration} / 2.0)) / 10000.0$$

And lastly we can output the results to the serial monitor.



Now write these lines of code into the “loop” function and upload it to test the ultrasonic sensor:

```
void loop() {
  float duration, distance; // duration in sec, distance in cm

  // Create a pulse
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); // Create a 10 microsec pulse
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH);

  // NB velocity of sound = 340 m/s; pulse travels there and back
  distance = (340.0 * (duration / 2.0)) / 10000.0;

  // Serial prints out results
  Serial.print(distance);
  Serial.println(" cm");
  delay(500); // Pause before sending another pulse
}
```

b. Piezoelectric Buzzer

A piezoelectric buzzer is one of the most simple audio output components, as it does not require any complicated circuit or any further software to setup.

All we need to do is write these lines of code just like any other common input or output component.

```
#define piezoPin 9

void setup() {
  pinMode(piezoPin, OUTPUT) // Set Buzzer as output
}
```

Then we will use the “map” function to limit the active distance from the ultrasonic sensor in this instance. The “map” function will divide the range of distance (represented by the first two integers), into the range of number of tone sequences available (represented by the last two integers).

After that we can use a simple “if.. else if” statement to forward to the corresponding tone sequence.

Now write these codes after the “Serial.print” output lines in the “loop” function:

```
int toneSeq = map(distance, 2, 30, 1, 3);
if (toneSeq == 1) {
  Twinkle();
}
else if (toneSeq == 2) {
  HappyBirthday();
}
else if (toneSeq == 2) {
  solitary();
}
```



And finally, copy these preset tone sequences:

```
// Tone Sequence 1: Twinkle Twinkle Little Star, by W. A. Mozart
void Twinkle() {
  // Bar 1
  tone(piezoPin, 262, 1000);
  delay(1000);
  tone(piezoPin, 262, 1000);
  delay(1000);
  tone(piezoPin, 392, 1000);
  delay(1000);
  tone(piezoPin, 392, 1000);
  delay(1000);
  tone(piezoPin, 440, 1000);
  delay(1000);
  tone(piezoPin, 440, 1000);
  delay(1000);
  tone(piezoPin, 392, 1000);
  delay(2000);

  // -----
  // Bar 2
  tone(piezoPin, 349, 1000);
  delay(1000);
  tone(piezoPin, 349, 1000);
  delay(1000);
  tone(piezoPin, 330, 1000);
  delay(1000);
  tone(piezoPin, 330, 1000);
  delay(1000);
  tone(piezoPin, 294, 1000);
  delay(1000);
  tone(piezoPin, 294, 1000);
  delay(1000);
  tone(piezoPin, 262, 1000);
  delay(2000);
}

// Tone Sequence 2: Happy Birthday To You, by P. Hill & M. J. Hill
void HappyBirthday() {
  // Bar 1
  tone(piezoPin, 294, 500);
  delay(500);
  tone(piezoPin, 294, 250);
  delay(250);
  tone(piezoPin, 324, 1000);
  delay(1000);
  tone(piezoPin, 294, 1000);
  delay(1000);
  tone(piezoPin, 392, 1000);
  delay(1000);
  tone(piezoPin, 370, 1000);
  delay(2000);

  // -----
  // Bar 2
  tone(piezoPin, 294, 500);
  delay(500);
  tone(piezoPin, 294, 250);
  delay(250);
}
```




```
tone(piezoPin, 324, 1000);
delay(1000);
tone(piezoPin, 294, 1000);
delay(1000);
tone(piezoPin, 440, 1000);
delay(1000);
tone(piezoPin, 392, 1000);
delay(2000);

// -----
// Bar 3
tone(piezoPin, 294, 500);
delay(500);
tone(piezoPin, 294, 250);
delay(250);
tone(piezoPin, 584, 1000);
delay(1000);
tone(piezoPin, 494, 1000);
delay(1000);
tone(piezoPin, 392, 500);
delay(500);
tone(piezoPin, 392, 250);
delay(250);
tone(piezoPin, 370, 1000);
delay(1000);
tone(piezoPin, 324, 1000);
delay(1000);

// -----
// Bar 4
tone(piezoPin, 523, 500);
delay(500);
tone(piezoPin, 523, 250);
delay(250);
tone(piezoPin, 494, 1000);
delay(1000);
tone(piezoPin, 392, 1000);
delay(1000);
tone(piezoPin, 440, 1000);
delay(1000);
tone(piezoPin, 392, 1000);
delay(1000);
}

// Tone Sequence 3: solitary in Eb, by H. Li
void solitary() {
  // Bar 1
  tone(piezoPin, 466, 469);
  delay(469);
  tone(piezoPin, 415, 469);
  delay(469);

  tone(piezoPin, 392, 469);
  delay(469);
  tone(piezoPin, 349, 469);
  delay(469);
  tone(piezoPin, 392, 469);
  delay(469);
  tone(piezoPin, 349, 469);
  delay(469);
  tone(piezoPin, 392, 938);
}
```



```
delay(938+469);

tone(piezoPin, 466, 469);
delay(469);
tone(piezoPin, 349, 938*2);
delay(938*3);

// -----
// Bar 2
tone(piezoPin, 466, 469);
delay(469);
tone(piezoPin, 415, 469);
delay(469);

tone(piezoPin, 392, 469);
delay(469);
tone(piezoPin, 349, 469);
delay(469);
tone(piezoPin, 392, 469);
delay(469);
tone(piezoPin, 311, 469);
delay(469);
tone(piezoPin, 349, 938);
delay(938+469);

tone(piezoPin, 466, 469);
delay(469);
tone(piezoPin, 587, 469);
delay(469*2);
}
```

Your final code should look like this:

```
#define trigPin 13
#define echoPin 12
#define piezoPin 9

void setup() {
  Serial.begin(9600); // Needed for Serial Monitor
  pinMode(trigPin, OUTPUT); // Set Trigger as output
  pinMode(echoPin, INPUT); // Set Echo as input
  pinMode(piezoPin, OUTPUT) // Set Buzzer as output
}

// Tone Sequence 1: Twinkle Twinkle Little Star, by W. A. Mozart
void Twinkle() {
  // Bar 1
  tone(piezoPin, 262, 1000);
  delay(1000);
  tone(piezoPin, 262, 1000);
  delay(1000);
  tone(piezoPin, 392, 1000);
  delay(1000);
  tone(piezoPin, 392, 1000);
  delay(1000);
  tone(piezoPin, 440, 1000);
  delay(1000);
  tone(piezoPin, 440, 1000);
}
```



```
delay(1000);
tone(piezoPin, 392, 1000);
delay(2000);

// -----
// Bar 2
tone(piezoPin, 349, 1000);
delay(1000);
tone(piezoPin, 349, 1000);
delay(1000);
tone(piezoPin, 330, 1000);
delay(1000);
tone(piezoPin, 330, 1000);
delay(1000);
tone(piezoPin, 294, 1000);
delay(1000);
tone(piezoPin, 294, 1000);
delay(1000);
tone(piezoPin, 262, 1000);
delay(2000);
}

// Tone Sequence 2: Happy Birthday To You, by P. Hill & M. J. Hill
void HappyBirthday() {
  // Bar 1
  tone(piezoPin, 294, 500);
  delay(500);
  tone(piezoPin, 294, 250);
  delay(250);
  tone(piezoPin, 324, 1000);
  delay(1000);
  tone(piezoPin, 294, 1000);
  delay(1000);
  tone(piezoPin, 392, 1000);
  delay(1000);
  tone(piezoPin, 370, 1000);
  delay(2000);

  // -----
  // Bar 2
  tone(piezoPin, 294, 500);
  delay(500);
  tone(piezoPin, 294, 250);
  delay(250);
  tone(piezoPin, 324, 1000);
  delay(1000);
  tone(piezoPin, 294, 1000);
  delay(1000);
  tone(piezoPin, 440, 1000);
  delay(1000);
  tone(piezoPin, 392, 1000);
  delay(2000);

  // -----
  // Bar 3
  tone(piezoPin, 294, 500);
  delay(500);
  tone(piezoPin, 294, 250);
  delay(250);
  tone(piezoPin, 584, 1000);
  delay(1000);
}
```



```
tone(piezoPin, 494, 1000);
delay(1000);
tone(piezoPin, 392, 500);
delay(500);
tone(piezoPin, 392, 250);
delay(250);
tone(piezoPin, 370, 1000);
delay(1000);
tone(piezoPin, 324, 1000);
delay(1000);

// -----
// Bar 4
tone(piezoPin, 523, 500);
delay(500);
tone(piezoPin, 523, 250);
delay(250);
tone(piezoPin, 494, 1000);
delay(1000);
tone(piezoPin, 392, 1000);
delay(1000);
tone(piezoPin, 440, 1000);
delay(1000);
tone(piezoPin, 392, 1000);
delay(1000);
}

// Tone Sequence 3: solitary in Eb, by H. Li
void solitary() {
  // Bar 1
  tone(piezoPin, 466, 469);
  delay(469);
  tone(piezoPin, 415, 469);
  delay(469);

  tone(piezoPin, 392, 469);
  delay(469);
  tone(piezoPin, 349, 469);
  delay(469);
  tone(piezoPin, 392, 469);
  delay(469);
  tone(piezoPin, 349, 469);
  delay(469);
  tone(piezoPin, 392, 938);
  delay(938+469);

  tone(piezoPin, 466, 469);
  delay(469);
  tone(piezoPin, 349, 938*2);
  delay(938*3);

  // -----
  // Bar 2
  tone(piezoPin, 466, 469);
  delay(469);
  tone(piezoPin, 415, 469);
  delay(469);

  tone(piezoPin, 392, 469);
  delay(469);
  tone(piezoPin, 349, 469);
```



```
    delay(469);
    tone(piezoPin, 392, 469);
    delay(469);
    tone(piezoPin, 311, 469);
    delay(469);
    tone(piezoPin, 349, 938);
    delay(938+469);

    tone(piezoPin, 466, 469);
    delay(469);
    tone(piezoPin, 587, 469);
    delay(469*2);
}

void loop() {
    float duration, distance; // duration in sec, distance in cm

    // create a pulse
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH); // Create a 10 microsec pulse
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);

    duration = pulseIn(echoPin, HIGH);

    // NB velocity of sound = 340 m/s; pulse travels there and back
    distance = (340.0 * (duration / 2.0)) / 10000.0;

    // Serial print out results
    Serial.print(distance);
    Serial.println(" cm");

    int toneSeq = map(distance, 2, 30, 1, 3);
    if (toneSeq == 1) {
        Twinkle();
    }
    else if (toneSeq == 2) {
        HappyBirthday();
    }
    else if (toneSeq == 3) {
        solitary();
    }
    delay(500); // Pause before sending another pulse
}
```



Customising A Musical Tone Sequence

If you would like to create a customised tone sequence, please follow this section and the template below.

```
void Title() {
  tone(piezoPin, Frequency, Note Duration);
  delay(Note Duration + Notes Intervals);
}
```

All you need to do is rename "Title" to the name of your sequence, then copy the "tone" and "delay" command. For the frequency of each note, please refer to Figure 3. Please ensure that both the frequency and note duration must be an integer, round all data from Figure 3 to the nearest digit.

NOTE	OCTAVE 0	OCTAVE 1	OCTAVE 2	OCTAVE 3	OCTAVE 4	OCTAVE 5	OCTAVE 6	OCTAVE 7	OCTAVE 8
C	16.35 Hz	32.70 Hz	65.41 Hz	130.81 Hz	A piano middle C 261.63 Hz	523.25 Hz	1046.50 Hz	2093.00 Hz	A piano's highest note 4186.01 Hz
C#/D♭	17.32 Hz	34.65 Hz	69.30 Hz	138.59 Hz	277.18 Hz	554.37 Hz	1108.73 Hz	2217.46 Hz	4434.92 Hz
D	18.35 Hz	36.71 Hz	73.42 Hz	146.83 Hz	293.66 Hz	587.33 Hz	1174.66 Hz	2349.32 Hz	4698.63 Hz
D#/E♭	19.45 Hz	38.89 Hz	77.78 Hz	155.56 Hz	311.13 Hz	622.25 Hz	1244.51 Hz	2489.02 Hz	4978.03 Hz
E	20.60 Hz	A bass's lowest note 41.20 Hz	A guitar's lowest note 82.41 Hz	164.81 Hz	329.63 Hz	659.25 Hz	1318.51 Hz	2637.02 Hz	5274.04 Hz
F	21.83 Hz	43.65 Hz	87.31 Hz	174.61 Hz	349.23 Hz	698.46 Hz	1396.91 Hz	2793.83 Hz	5587.65 Hz
F#/G♭	23.12 Hz	46.25 Hz	92.50 Hz	185.00 Hz	369.99 Hz	739.99 Hz	1479.98 Hz	2959.96 Hz	5919.91 Hz
G	24.50 Hz	49.00 Hz	98.00 Hz	A violin's lowest note 196.00 Hz	392.00 Hz	783.99 Hz	1567.98 Hz	3135.96 Hz	6271.93 Hz
G#/A♭	25.96 Hz	51.91 Hz	103.83 Hz	207.65 Hz	415.30 Hz	830.61 Hz	1661.22 Hz	3322.44 Hz	6644.88 Hz
A	A piano's lowest note 27.50 Hz	55.00 Hz	110.00 Hz	220.00 Hz	440.00 Hz	880.00 Hz	1760.00 Hz	3520.00 Hz	7040.00 Hz
A#/B♭	29.14 Hz	58.27 Hz	116.54 Hz	233.08 Hz	466.16 Hz	932.33 Hz	1864.66 Hz	3729.31 Hz	7458.62 Hz
B	A 5 string bass's lowest note 30.87 Hz	61.74 Hz	123.47 Hz	246.94 Hz	493.88 Hz	987.77 Hz	1975.53 Hz	3951.07 Hz	7902.13 Hz

Fig.6: Music Note To Frequency Chart. ^[2]

After creating your tone sequence, change the last integer (x) of the "map" function,

```
int toneSeq = map(distance, 2, 30, 1, x);
```

and then copy these lines below into the if statement at the end of the code, and change "x" to the next number of the "if" statement, and lastly change "Title" to the name of your sequence.

```
else if (toneSeq == x) {
  Title();
}
```

Now verify and upload your code to the Arduino, ask a member of the committees if you have any questions.



References

- [1] Autodesk Instructables, Ultrasonic Sensor HC-SR04 (Arduino Project) : 5 Steps (with Pictures), <https://www.instructables.com/Ultrasonic-Sensor-HC-SR04-Arduino-Project/>
- [2] MixButton, Music Note Frequency Chart - Music Frequency Chart, <https://mixbutton.com/mixing-articles/music-note-to-frequency-chart/>

Original Author: Henry Li, (sec 24/25)

First published: 17th January 2025

Last revised & edited by: Henry Li

Last revised & edited: 19th January 2025

Version: 3

© 2025 Makers' Society of RHSU & Royal Holloway, University of London.